

Module 5 Report: Software Assurance and Secure SQL

Grad Café Analytics · Khaled Al-Hassan (kalthass2) · JHU EP 605.256

Repository: `git@github.com:khalidalhassan10/jhu_software_concepts.git` (folder `module_5/`)

1. Summary of results

Requirement	Result	Evidence
Pylint on <code>module_5/src</code>	10.00/10	<code>pylint_report.txt</code> ; CI job fails below 10
Tests	135 passed, 100% coverage	<code>coverage_summary.txt</code> ; CI pytest job
SQL injection defences	All queries composed with <code>psycopg.sql</code> , parameters bound, LIMIT on every query	Section 3; <code>tests/test_sql_safety.py</code>
Least-privilege database user	<code>gradcafe_app</code> : SELECT and INSERT only	Section 4; <code>db/least_privilege.sql</code>
Dependency graph	Generated with <code>pydeps</code> and <code>Graphviz</code>	<code>dependency.svg</code> ; Section 5
Reproducible install	<code>pip</code> and <code>uv</code> both verified in a fresh virtual environment	<code>requirements.txt</code> , <code>setup.py</code> , README
Snyk Open Source	31 dependencies, 0 vulnerabilities	<code>snyk-analysis.png</code>
Snyk Code (extra credit)	0 high, 0 medium, 1 low (reviewed)	<code>snyk-code-test.png</code> ; Section 7
GitHub Actions	4 jobs, all passing	<code>.github/workflows/ci.yml</code> ; <code>actions_success.png</code>

2. How to install and run

Requirements: Python 3.12 and PostgreSQL 16 or newer. All commands run from `module_5`. Both methods were tested from scratch in a brand-new virtual environment, and the full test suite passed in each.

Option 1: pip

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
pip install -e .
```

Option 2: uv

```
python3.12 -m pip install uv
uv venv --python 3.12 .venv
source .venv/bin/activate
uv pip sync requirements.txt
uv pip install -e .
```

`uv pip sync` makes the environment match `requirements.txt` exactly: it installs what is listed and removes anything that is not. `requirements.in` lists the nine packages the project uses directly, and `requirements.txt` pins all 33 packages to exact versions. It was generated with `uv pip compile requirements.in --universal --python-version 3.12`.

Database, configuration and running

```
createdb -U postgres gradcafe
createdb -U postgres gradcafe_test
export DATABASE_URL="postgresql://postgres:PASSWORD@localhost:5432/gradcafe"
python -m src.load_data ../module_3/llm_extend_applicant_data.json
psql "$DATABASE_URL" -f db/least_privilege.sql
psql "$DATABASE_URL" -c "\password gradcafe_app"
```

```
cp .env.example .env
set -a; source .env; set +a
unset DATABASE_URL
flask --app src.flask_app:create_app run
```

`load_data` creates the table and loads the rows. The two `psql` lines create the `gradcafe_app` user and set its password. In `.env`, the real `gradcafe_app` password replaces the placeholder. After `unset DATABASE_URL`, the app connects as `gradcafe_app`, and the page is at <http://127.0.0.1:5000/analysis>.

Tests, from the repository root: `pytest module_5/tests -m "web or buttons or analysis or db or integration"`, with `TEST_DATABASE_URL` pointing at a database whose name ends in `_test`.

3. SQL injection defences

What changed. In Module 4, the analysis queries were hand-written strings with the table name and values typed into the SQL text, and none had a `LIMIT`. In Module 5, every raw SQL statement run through `psycpg` is built with `psycpg`'s composition tools; the ORM queries are `SQLAlchemy` expressions, which also send values as bound parameters, and the only fixed SQL string left is the `CREATE TABLE` statement, which contains no input. A new search endpoint, `GET /api/applicants` (the first place where text typed by a user reaches SQL), was added on top of them.

- `sql.SQL` holds the fixed SQL text, written only by the developer.

- `sql.Identifier` supplies table and column names. `psycpg` double-quotes them, so a name can never become SQL code.
- `%s` placeholders (`sql.Placeholder`) stand for every value. The values travel separately, in `execute(statement, params)`.
- The sort column is checked against an allow-list (`SORTABLE_COLUMNS`). Anything else falls back to `p_id`, so user text is only compared, never inserted.
- Filter values are wrapped by `contains_pattern`, which escapes the user's own `%`, `_` and `\`, so a lone `%` cannot match every row.
- Requested limits go through `clamp_limit`, which turns any input into a whole number from 1 to 100 (default 20).

The search statement is built by `build_search`, which only constructs it, and run by `search_applicants`:

```
statement = sql.SQL("SELECT {columns} FROM {table}{where} ORDER BY {sort} LIMIT %s").format(
    columns=sql.SQL(", ").join(sql.Identifier(name) for name in RESULT_COLUMNS),
    table=TABLE, where=where, sort=sql.Identifier(sort_column))
params.append(clamp_limit(limit))
return statement, params
```

For `term=Fall 2026`, `degree=Masters`, `sort=gpa`, `limit=5`, this produces:

```
SELECT "p_id", ..., "date_added" FROM "applicants"
WHERE "term" ILIKE %s AND "degree" ILIKE %s ORDER BY "gpa" LIMIT %s
params = ['%Fall 2026%', '%Masters%', 5]
```

Why it is safe. The SQL text that reaches PostgreSQL contains only words the developer wrote. Everything a user typed is in the parameter list, and PostgreSQL treats parameters strictly as data, never as SQL. A test builds a search and asserts that the user's text does not appear in the SQL string. The endpoint also returns a fixed error message when the database fails, so internal details never leak. Pull Data and Update Analysis behave the same way: they return fixed messages and write the details only to the server log.

Attack sent as a filter or parameter	Result (tested in <code>tests/test_sql_safety.py</code>)
<code>' OR 1=1 --</code>	0 rows: treated as text, and no term contains it
<code>x'; DROP TABLE applicants; --</code>	0 rows, and the table still holds all its rows
<code>Fall 2026' UNION SELECT username FROM pg_user --</code>	0 rows, and no user list is read
<code>% and _</code>	0 rows: escaped, they match only themselves
<code>sort=gpa; DROP TABLE applicants</code>	not on the allow-list, so sorted by <code>p_id</code> ; table intact
<code>limit=100000, limit=abc, limit=0</code>	clamped to 100, 20 and 1 rows

Each attack runs against a test table of 105 rows, both through the Python function and through the web endpoint, and every test also checks that the table still contains 105 rows afterwards. A normal search for a real term is tested first, so the attack tests cannot pass merely because the search finds nothing.

Requirements met for SQL

- **LIMIT enforced:** every query has one.
 - The eleven analysis queries use `LIMIT %s` with the value 1, since each returns one row.
 - The row count uses `LIMIT 1`, and the ORM queries add `.limit(1)` through two helper functions.
 - Pull Data asks only about the ids on each results page, in batches of at most 100 (`WHERE p_id = ANY(%s) LIMIT %s`) instead of loading all stored ids.
 - The search is clamped to at most 100 rows.
- **Statements separated from execution:** statements are composed once, as module-level constants or by builder functions (`build_search`, `average_of`), and executed separately with their parameters.
- **Safe composition and parameterization:** `sql.SQL`, `sql.Identifier` and placeholders are used throughout. No f-strings or string concatenation build SQL anywhere in `src`.

4. Least-privilege database configuration

No credentials in code. `db_config.py` reads `DATABASE_URL`. If that isn't set, it builds the URL from `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER` and `DB_PASSWORD`. The user and password are URL-encoded, so characters such as `@`, `/` and `:` are safe. `.env.example` lists the variables with placeholder values, and the real `.env` is listed in `.gitignore`, and has never been committed to the repository.

The app's user. The app connects as `gradcafe_app`, created by `db/least_privilege.sql`:

```
CREATE ROLE gradcafe_app
LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOREPLICATION NOBYPASSRLS;
REVOKE ALL ON DATABASE gradcafe FROM PUBLIC;
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_app;
GRANT USAGE ON SCHEMA public TO gradcafe_app;
GRANT SELECT, INSERT ON TABLE applicants TO gradcafe_app;
```

Permission	Why
LOGIN, with no superuser, create-database, create-role, replication or bypass-RLS powers	It must connect, but it needs none of the administrative powers. PostgreSQL's default PUBLIC CONNECT and TEMPORARY on gradcafe are revoked first, so only roles granted CONNECT can connect, and gradcafe_app cannot create even temporary tables there. (Like every role, it can still connect to other databases
CONNECT on database gradcafe	

Permission

Why

USAGE on schema public	that keep the default PUBLIC connect privilege, but it has no table privileges there.).
SELECT on applicants	It may see the objects in the schema. This does not allow creating anything.
INSERT on applicants	The analysis page, Update Analysis and the search read rows.
Not granted: UPDATE, DELETE, TRUNCATE, DROP, ALTER, CREATE	Pull Data adds new rows. ON CONFLICT DO NOTHING needs only INSERT.
	The app never changes, removes or restructures data, so a compromised app could not either.

Because the app may not create tables, table creation moved out of the request path. `insert_rows` and the analysis query no longer run `CREATE TABLE IF NOT EXISTS;` instead, the owner creates the table once with `python -m src.load_data`.

Testing every kind of statement as `gradcafe_app` confirmed that SELECT and INSERT succeed, while UPDATE, DELETE, TRUNCATE, DROP, ALTER, CREATE TABLE and CREATE TEMP TABLE are all refused. The full application was then run with only the five `DB_*` variables (with `DATABASE_URL` unset), and the analysis page loaded the real data.

The tests deliberately keep using the owner account on the separate `gradcafe_test` database, because they must empty the table between tests.

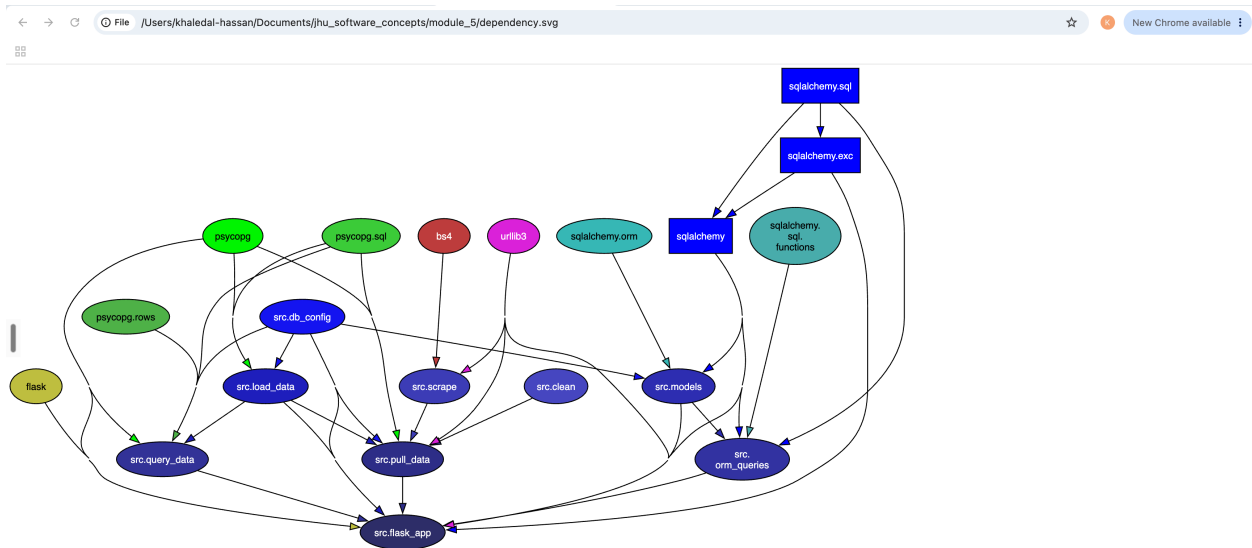
```
-----
Password for user postgres:
List of roles
Role name | Attributes
-----+-----
gradcafe_app |
```

```
Password for user postgres:
privilege_type
-----
INSERT
SELECT
(2 rows)
```

```
(py312) khaledal-hassan@MacBookPro module_5 % █
```

5. Dependency graph

The graph was generated with `pydeps src --noshow -T svg -o dependency.svg` (Graphviz provides the dot renderer) and saved as `module_5/dependency.svg`. Arrows point from a module to the module that imports it.



The graph shows that `src.flask_app` is the hub of the project: it imports `query_data` (the safe search), `orm_queries` (the eleven answers on the analysis page), `pull_data` (the Pull Data button), `load_data` (inserting rows), `models` and `db_config`, and it builds the web layer with Flask. `src.pull_data` is the Pull Data pipeline: it chains `scrape` (which fetches pages with `urllib3` and parses them with `BeautifulSoup`, `bs4`), `clean` (which tidies each entry) and `load_data` (which stores the new rows). `src.db_config` is imported by every module that talks to the database (`load_data`, `query_data`, `pull_data`, `models` and `flask_app`), so the connection settings from the environment and the row limits live in exactly one place. The raw-SQL modules (`load_data`, `query_data` and `pull_data`) depend on `psycopg` and `psycopg.sql`, which provide parameter binding and the safe SQL composition (SQL, Identifier, Placeholder) used against SQL injection, while `query_data` also uses `psycopg.rows` to return search results as dictionaries. The ORM side, `models` and `orm_queries`, depends on SQLAlchemy (`sqlalchemy.orm` for the Applicant model, `sqlalchemy.sql.functions` for counts), and `flask_app` imports `sqlalchemy.exc` only to catch database errors. There are no circular imports: every arrow flows downward from small helper modules toward `flask_app`, which keeps each part testable on its own.

6. Packaging: why setup.py matters

`setup.py` describes the project as an installable package named `gradcafe-analytics`, version 5.0.0. It declares the `src` package, includes the HTML template and CSS, requires Python 3.12 or newer, lists the five runtime libraries in `install_requires`, and lists the development tools (`pytest`, `pytest-cov`, `pylint`, `pydeps`) as a `dev extra`.

- **Consistent imports:** `pip install -e .` installs the project into the environment, so `import src.flask_app` works the same in a terminal, in the tests and in CI, regardless of the current folder. This removes the path problems behind "it works on my machine".
- **Editable development:** the editable install points at the source folder, so code changes take effect immediately, without reinstalling.
- **Reproducibility with tools:** packaging metadata is what `pip` and `uv` read to install the project. Together with the exact pins in `requirements.txt`, every machine (mine, the grader's and GitHub Actions) builds the same environment. The CI `pytest` job installs the project with `pip install -e module_5`.

7. Snyk scans

Dependency scan (required)

`snyk test --file=requirements.txt --package-manager=pip` checked all 31 installed dependencies against Snyk's vulnerability database and found no vulnerable paths, so no package needed patching or removal. The same scan runs in CI and fails the build if a vulnerability ever appears.

```
10.      40m/snyk.interaction/0/51d930-5001-4000-0000-001273047010
● ((.venv) ) (py312) khaledal-hassan@MacBookPro jhu_software_concepts % cd module_5
snyk test --file=requirements.txt --package-manager=pip
```

Testing (requirements.txt) ...

✓ Tested 31 dependencies for known issues, no vulnerable paths found.

Test Summary

Organization:	khalidalhassan10
Test type:	open-source
Project path:	/Users/khaledal-hassan/Documents/jhu_software_concepts/module_5
Total issues:	0

💡 Tip

To view ignored issues, use the `--include-ignores` option.

```
○ ((.venv) ) (py312) khaledal-hassan@MacBookPro module_5 %
```

Snyk Code (SAST, extra credit)

snky code test first reported 3 issues in my own code. Two were MEDIUM and are now fixed:

1. **Debug Mode Enabled:** flask_app.py started Flask with debug=True, which exposes an interactive debugger that can run code on the server. It now runs with debug=False, and a test checks this.
2. **Server Information Exposure:** when the database could not be read, the analysis page displayed the raw exception text, which can reveal internal details such as the database host or user. The page now shows a fixed message, the full error is written only to the server log, and a test checks that the internal text never reaches the page.

The remaining LOW finding, **Path Traversal** in load_data.py, flags that the loading script opens a file path given on the command line. I reviewed it and accepted it as a false positive. This is an administrator setup script, run only by the database owner on their own machine with a file of their choosing (for example, ../module_3/llm_extend_applicant_data.json). There is no untrusted user who could supply the path, and restricting it to one folder would break the documented setup command.

The rescan shows 0 HIGH, 0 MEDIUM and 1 LOW issue. In CI, snky code test --severity-threshold=medium fails the build on any new medium or high finding.

Open Issues

```
x [LOW] Path Traversal
Finding ID: ce8fb87f-c706-4c20-88a2-2366fd837ad6
Path: src/load_data.py, line 193
Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This
may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.
```

Test Summary

Organization:	khalidalhassan10
Test type:	Static code analysis
Project path:	/Users/khaledal-hassan/Documents/jhu_software_concepts/module_5
Total issues:	1
Ignored issues:	0 [0 HIGH 0 MEDIUM 0 LOW]
Open issues:	1 [0 HIGH 0 MEDIUM 1 LOW]

Tip

To view ignored issues, use the --include-ignores option.

```
❖ ((.venv) ) (py312) khaledal-hassan@MacBookPro module_5 %
```

8. CI enforcement with GitHub Actions

.github/workflows/ci.yml is at the repository root, where GitHub runs it, with a copy in module_5/.github/workflows/. It runs on every push and pull request, with four separate jobs, and the workflow's access to the repository is limited to read-only (contents: read):

- **Pylint:** installs the pinned requirements and runs `pylint src --fail-under=10`.
- **Dependency graph:** installs Graphviz, generates `dependency.svg` with `pydeps`, fails if the file is missing or empty (`test -s`), and uploads it as an artifact.
- **Snyk:** runs `snyk test` on the dependencies (failing on known vulnerabilities) and `snyk code test --severity-threshold=medium`, authenticated with the `SNYK_TOKEN` repository secret.
- **Pytest:** starts a PostgreSQL 16 service, installs the project, and runs the whole suite, which fails below 100% coverage.

← ci

✓ **Module 5: secure SQL, least privilege, packaging, Snyk, CI #1**

Re-run all jobs

...

Summary

All jobs

- ✓ Pylint (must be 10/10)
- ✓ Dependency graph (pydeps + Graphviz)
- ✓ Snyk scans
- ✓ Pytest (100% coverage)

Run details

Usage

Workflow file

Triggered via push 2 minutes ago

Status

Total duration

Artifacts

✉ khalidhassan10 pushed → 5cf683b main

Success

47s

1

ci.yml

on: push

- ✓ Pylint (must be 10/10) 29s
- ✓ Dependency graph (pyde... 33s
- ✓ Snyk scans 43s
- ✓ Pytest (100% coverage) 34s

⌂ - +